

Day 3: Basic Sorting Algorithms

Introduction to Sorting Techniques

Sorting means rearranging a list/array of elements according to a defined comparison rule (e.g., ascending numbers, alphabetical strings). The rule tells the algorithm how to decide the new order of elements.

Why Sorting Algorithms Matter

Sorting algorithms simplify complex problems and improve efficiency. They are used in:

- Searching (especially binary search)
- Organizing large datasets
- Quickly finding the smallest or largest element
- Pre-processing data for advanced algorithms (divide & conquer, graph algorithms, etc.)

1. Selection Sort

Selection Sort repeatedly finds the smallest (or largest) element from the unsorted part of the array and places it at the correct position in the sorted part.

Space Inefficient Selection Sort

Creates a new array for the sorted values.

```
def selectionsort(unsorted):
```

```
    sorted_ = []
```

```
    while len(unsorted) > 1:
```

```
        min_ = float('inf')
```

```
        for item in unsorted:
```

```

        if item < min_:
            min_ = item

    sorted_.append(min_)
    unsorted.remove(min_)

return sorted_

arr = [23, 1, 45, 32, 90, 87, 78, 78]
print(selectionsort(arr))
# Output: [1, 23, 32, 45, 78, 78, 87, 90]

```

Explanation:

- We repeatedly find the minimum element and move it into a new list.
- This is easy to understand but not space efficient.

Complexity:

- Time: $O(n^2)$ for all cases (inner loop always runs).
- Space: $O(n)$ extra space for the new array.

Use Case: When code clarity matters more than memory usage and dataset is small.

Space Efficient Selection Sort

Sorts the array in-place by swapping indexes.

```

def selectionsort2(unsorted):
    for j in range(len(unsorted)):
        min_ = float('inf')

```

```
    for i in range(j, len(unsorted)):
        if unsorted[i] < min_:
            min_ = unsorted[i]
            index = i
    unsorted[index] = unsorted[j]
    unsorted[j] = min_

return unsorted

arr = [23, 1, 45, 32, 90, 87, 78]
print(selectionsort2(arr))
# Output: [1, 23, 32, 45, 78, 87, 90]
```

Explanation:

- Instead of creating a new array, we swap the found minimum with the first unsorted position.

Complexity:

- Time: $O(n^2)$ for all cases.
- Space: $O(1)$ extra space.

Use Case: When memory is limited but swaps are acceptable.

2. Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After each pass, the largest unsorted element “bubbles up” to its correct position at the end.

```
def bubblesort(unsorted):
    swapped = True
    while swapped:
        swapped = False
        for i in range(len(unsorted) - 1):
            if unsorted[i] > unsorted[i + 1]:
                unsorted[i], unsorted[i + 1] = unsorted[i + 1], unsorted[i]
                swapped = True
    return unsorted

arr = [23, 1, 45, 32, 90, 87, 78]
print(bubblesort(arr))
# Output: [1, 23, 32, 45, 78, 87, 90]
```

Explanation:

- Each pass moves the largest remaining element to its correct place.
- With the swapped flag, the algorithm stops early if no swaps happen.

Complexity:

- Best Case: $O(n)$ (already sorted array).
- Worst/Average Case: $O(n^2)$.
- Space: $O(1)$.

Use Case: Educational/teaching algorithm. Not efficient for large arrays but useful when the data is almost sorted.

3. Insertion Sort

Insertion Sort builds the final sorted array one element at a time. It takes the next element and inserts it into its correct position within the sorted portion.

```
def insertionsort(unsorted):  
    for i in range(1, len(unsorted)):  
        j = i - 1  
        target = unsorted[i]  
        while j >= 0:  
            if unsorted[j] > target:  
                unsorted[j + 1] = unsorted[j]  
                unsorted[j] = target  
                j = j - 1  
            else:  
                break  
    return unsorted
```

```
arr = [23, 1, 45, 32, 90, 87, 78, 78]  
print(insertionsort(arr))  
# Output: [1, 23, 32, 45, 78, 78, 87, 90]
```

Explanation:

- Works like sorting cards in your hand.
- Shifts larger elements to the right until the correct position for the current element is found.

Complexity:

- Best Case: $O(n)$ (array already sorted).
- Worst/Average Case: $O(n^2)$.

- Space: $O(1)$.

Use Case: Small datasets or datasets that are already nearly sorted. It's stable and simple.

Summary Table: Comparison of Basic Sorting Algorithms

Algorithm	Best Time	Worst Time	Space	Stable	When to Use
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$	No	Small arrays, when minimizing swaps is important
Bubble Sort	$O(n)$	$O(n^2)$	$O(1)$	Yes	Teaching, very small arrays, nearly sorted data
Insertion Sort	$O(n)$	$O(n^2)$	$O(1)$	Yes	Small or mostly sorted arrays, stable sort required

How to Choose

- Use **Insertion Sort** if the data is small or nearly sorted and stability matters.
- Use **Selection Sort** if swaps are expensive and you want to minimize them.
- Avoid **Bubble Sort** for production (inefficient) but use it for teaching concepts or extremely small datasets